

```
# Upload your Kaggle token
from google.colab import files
files.upload() # Upload kaggle.json here

# The file kaggle.json already exists in /content/ so no need to upload it again.
```

No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving kaggle.json to kaggle.json

```
{'kaggle.json': b'{"username": "sanskriti2407" "key": "2e0d8ef8d4565723a020bb9c743fe4a3"}'}
```

```
# Setup Kaggle API
!mkdir -p ~/.kaggle
!mv kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json
!pip install -q kaggle
```

```
# Download the German Traffic Sign Recognition dataset
!kaggle datasets download -d meowmeowmeowmeowmeow/gtsrb-german-traffic-sign
!unzip -q gtsrb-german-traffic-sign.zip -d traffic_sign_data
```

Dataset URL: <https://www.kaggle.com/datasets/meowmeowmeowmeowmeow/gtsrb-german-traffic-sign>
 License(s): CC0-1.0
 Downloading gtsrb-german-traffic-sign.zip to /content
 89% 543M/612M [00:03<00:01, 70.7MB/s]
 100% 612M/612M [00:03<00:00, 190MB/s]

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import os
import cv2
from sklearn.model_selection import train_test_split
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import EarlyStopping
```

```
data = []
labels = []
num_classes = 43 # Total traffic sign classes
data_dir = '/content/traffic_sign_data/Train'

# Loop through each class folder
for i in range(num_classes):
    path = os.path.join(data_dir, str(i))
    images = os.listdir(path)
    for img_name in images:
        try:
            img = cv2.imread(os.path.join(path, img_name))
            img = cv2.resize(img, (30, 30))
            data.append(img)
            labels.append(i)
        except:
            print("Error loading image:", img_name)

# Convert to numpy arrays
data = np.array(data)
labels = np.array(labels)

print("Data shape:", data.shape)
print("Labels shape:", labels.shape)
```

Data shape: (39209, 30, 30, 3)
 Labels shape: (39209,)

```
# Split into train and test sets
X_train, X_test, y_train, y_test = train_test_split(
    data, labels, test_size=0.2, random_state=42
)

# Normalize pixel values
X_train = X_train / 255.0
X_test = X_test / 255.0
```

```
# One-hot encode the labels
y_train = to_categorical(y_train, num_classes)
y_test = to_categorical(y_test, num_classes)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (31367, 30, 30, 3)
Testing data: (7842, 30, 30, 3)
```

```
model = Sequential([
    Conv2D(32, (5,5), activation='relu', input_shape=(30,30,3)),
    MaxPooling2D(pool_size=(2,2)),

    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D(pool_size=(2,2)),

    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(num_classes, activation='softmax')
])

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	2,432
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 43)	5,547

```
Total params: 231,403 (903.92 KB)
Trainable params: 231,403 (903.92 KB)
Non-trainable params: 0 (0.00 B)
```

```
datagen = ImageDataGenerator(
    rotation_range=10,
    zoom_range=0.1,
    width_shift_range=0.1,
    height_shift_range=0.1
)
datagen.fit(X_train)

# Early stopping for better convergence
early_stop = EarlyStopping(monitor='val_accuracy', patience=3, restore_best_weights=True)

history = model.fit(
    datagen.flow(X_train, y_train, batch_size=32),
    epochs=15,
    validation_data=(X_test, y_test),
    callbacks=[early_stop]
)
```

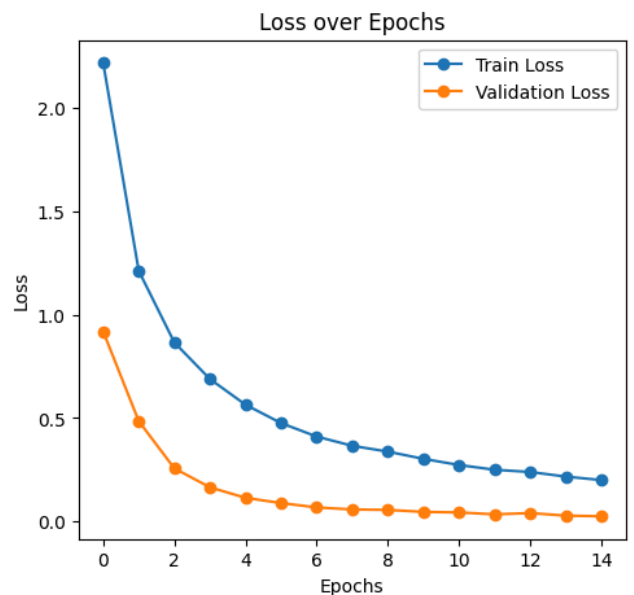
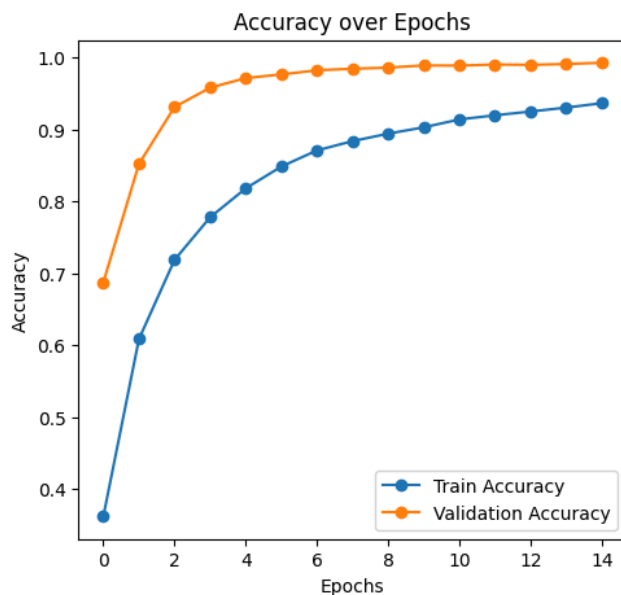
```
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDataset`
self._warn_if_super_not_called()
```

```
Epoch 1/15
981/981 ————— 61s 58ms/step - accuracy: 0.2334 - loss: 2.8342 - val_accuracy: 0.6875 - val_loss: 0.9202
Epoch 2/15
981/981 ————— 53s 54ms/step - accuracy: 0.5683 - loss: 1.3366 - val_accuracy: 0.8522 - val_loss: 0.4857
Epoch 3/15
981/981 ————— 53s 54ms/step - accuracy: 0.6962 - loss: 0.9382 - val_accuracy: 0.9314 - val_loss: 0.2571
Epoch 4/15
981/981 ————— 52s 53ms/step - accuracy: 0.7682 - loss: 0.7200 - val_accuracy: 0.9583 - val_loss: 0.1653
```

```
Epoch 5/15
981/981 ————— 54s 55ms/step - accuracy: 0.8099 - loss: 0.5929 - val_accuracy: 0.9717 - val_loss: 0.1143
Epoch 6/15
981/981 ————— 55s 56ms/step - accuracy: 0.8426 - loss: 0.4896 - val_accuracy: 0.9768 - val_loss: 0.0891
Epoch 7/15
981/981 ————— 57s 58ms/step - accuracy: 0.8657 - loss: 0.4256 - val_accuracy: 0.9825 - val_loss: 0.0678
Epoch 8/15
981/981 ————— 52s 53ms/step - accuracy: 0.8817 - loss: 0.3737 - val_accuracy: 0.9847 - val_loss: 0.0581
Epoch 9/15
981/981 ————— 52s 53ms/step - accuracy: 0.8911 - loss: 0.3515 - val_accuracy: 0.9862 - val_loss: 0.0559
Epoch 10/15
981/981 ————— 54s 55ms/step - accuracy: 0.8996 - loss: 0.3148 - val_accuracy: 0.9893 - val_loss: 0.0463
Epoch 11/15
981/981 ————— 52s 53ms/step - accuracy: 0.9124 - loss: 0.2795 - val_accuracy: 0.9890 - val_loss: 0.0437
Epoch 12/15
981/981 ————— 82s 53ms/step - accuracy: 0.9174 - loss: 0.2593 - val_accuracy: 0.9906 - val_loss: 0.0344
Epoch 13/15
981/981 ————— 82s 54ms/step - accuracy: 0.9250 - loss: 0.2406 - val_accuracy: 0.9901 - val_loss: 0.0405
Epoch 14/15
981/981 ————— 53s 54ms/step - accuracy: 0.9319 - loss: 0.2109 - val_accuracy: 0.9913 - val_loss: 0.0282
Epoch 15/15
981/981 ————— 82s 54ms/step - accuracy: 0.9358 - loss: 0.1979 - val_accuracy: 0.9930 - val_loss: 0.0250
```

```
plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train Accuracy', marker='o')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy', marker='o')
plt.title("Accuracy over Epochs")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()

plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train Loss', marker='o')
plt.plot(history.history['val_loss'], label='Validation Loss', marker='o')
plt.title("Loss over Epochs")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()
```



```
test_loss, test_acc = model.evaluate(X_test, y_test)
print(f"\n✅ Test Accuracy: {test_acc * 100:.2f}%")
```

```
246/246 ————— 3s 10ms/step - accuracy: 0.9940 - loss: 0.0253
```

```
✅ Test Accuracy: 99.30%
```

```
# Save trained model
model.save("traffic_sign_model.h5")

# Save label mapping
label_map = {
    "0": "Speed limit 20 km/h",
```

```

"1": "Speed limit 30 km/h",
"2": "Speed limit 50 km/h",
"3": "Speed limit 60 km/h",
"4": "Speed limit 70 km/h",
"5": "Speed limit 80 km/h",
"6": "End of speed limit 80 km/h",
"7": "Speed limit 100 km/h",
"8": "Speed limit 120 km/h",
"9": "No passing",
"10": "No passing for vehicles over 3.5 tons",
"11": "Right-of-way at the next intersection",
"12": "Priority road",
"13": "Yield",
"14": "Stop",
"15": "No vehicles",
"16": "Vehicles over 3.5 tons prohibited",
"17": "No entry",
"18": "General caution",
"19": "Dangerous curve left",
"20": "Dangerous curve right",
"21": "Double curve",
"22": "Bumpy road",
"23": "Slippery road",
"24": "Road narrows",
"25": "Construction",
"26": "Traffic signals",
"27": "Pedestrians",
"28": "Children crossing",
"29": "Bicycles crossing",
"30": "Beware of ice/snow",
"31": "Wild animals crossing",
"32": "End of all speed and passing limits",
"33": "Turn right ahead",
"34": "Turn left ahead",
"35": "Ahead only",
"36": "Go straight or right",
"37": "Go straight or left",
"38": "Keep right",
"39": "Keep left",
"40": "Roundabout mandatory",
"41": "End of no passing",
"42": "End of no passing by vehicles over 3.5 tons"
}

import json
with open("label_map.json", "w") as f:
    json.dump(label_map, f)

```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format

```

!pip install -q gradio
import gradio as gr
from tensorflow.keras.models import load_model
from PIL import Image

# Load the trained model and label map
model = load_model("/content/traffic_sign_model.h5")
with open("/content/label_map.json", "r") as f:
    label_map = json.load(f)

def classify_image(image):
    img = image.resize((30,30))
    img_array = np.array(img) / 255.0
    img_array = np.expand_dims(img_array, axis=0)
    predictions = model.predict(img_array)
    class_id = np.argmax(predictions)
    confidence = np.max(predictions) * 100
    sign_name = label_map.get(str(class_id), "Unknown Sign")
    return {"sign_name": sign_name, "confidence": float(confidence/100)}

# Build Gradio interface
interface = gr.Interface(
    fn=classify_image,
    inputs=gr.Image(type="pil"),
    outputs=gr.Label(num_top_classes=1),
    title="🚦 Traffic Sign Classifier",
    description="Upload a traffic sign image to see the predicted class",
    examples=[
        ["traffic_sign_data/Train/0/00000_00000_00000.png"],

```

```
        ["traffic_sign_data/Train/1/00001_00000_00000.png"]
    ]
)

interface.launch(share=True)
```

WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile\_metrics` will be empty until Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
* Running on public URL: <https://1cab9a17f52e3a2a0c.gradio.live>

This share link expires in 1 week. For free permanent hosting and GPU upgrades, run `gradio deploy` from the terminal in the workspace.



No interface is running right now



```
import os

data_dir = '/content/traffic_sign_data/Train' # Change to your training folder path
num_classes = 43

train_samples = {}

for i in range(num_classes):
    class_path = os.path.join(data_dir, str(i))
    if os.path.exists(class_path):
        train_samples[i] = len(os.listdir(class_path))
    else:
        train_samples[i] = 0

# Print sample count per class
for class_id, count in train_samples.items():
    print(f"Class {class_id}: {count} images")
```

```
Class 0: 210 images
Class 1: 2220 images
Class 2: 2250 images
Class 3: 1410 images
Class 4: 1980 images
Class 5: 1860 images
Class 6: 420 images
Class 7: 1440 images
Class 8: 1410 images
Class 9: 1470 images
Class 10: 2010 images
Class 11: 1320 images
Class 12: 2100 images
Class 13: 2160 images
Class 14: 780 images
Class 15: 630 images
Class 16: 420 images
Class 17: 1110 images
Class 18: 1200 images
Class 19: 210 images
Class 20: 360 images
Class 21: 330 images
Class 22: 390 images
Class 23: 510 images
Class 24: 270 images
```

```
Class 25: 1500 images
Class 26: 600 images
Class 27: 240 images
Class 28: 540 images
Class 29: 270 images
Class 30: 450 images
Class 31: 780 images
Class 32: 240 images
Class 33: 689 images
Class 34: 420 images
Class 35: 1200 images
Class 36: 390 images
Class 37: 210 images
Class 38: 2070 images
Class 39: 300 images
Class 40: 360 images
Class 41: 240 images
Class 42: 240 images
```

Start coding or generate with AI.

